Smart Traffic Congestion Prediction

Live demo walkthrough and learning guide
Computer vision + machine learning

How to use this document

Part A: Live Demo Walkthrough. Follow this while you present. It lists exactly what to click and say in the running app, with timings and fixes if something breaks.

Part B: Learning Guide. Read this before presenting so you can explain every part: the data, the models, the computer vision, the fusion, and the newer features (lessons 11 to 15), plus a 2-minute script and interview questions.

Part C: Concepts and Talking Points. Plain-language answers to "what is X?" and "how does Y work?": YOLO, CNN, OpenCV, OCR, how the three ML models are calculated and trained, the dataset, the decimal numbers and formulas, why California, what makes the project different, which cameras to use, and two scripts to read aloud.

Prepared 22 October 2026. The simulated challans are a demonstration, not real enforcement.

Contents

Part A: Live Demo Walkthrough 4

0. Before you present (do this 20 minutes early) . . . 4
1. Opening (30 seconds, on the header) . . . 4
2. Tab 1: Single camera (about 4 minutes) — must show . . . 4
3. Tab 2: Monitoring grid (1 minute) — must show . . . 5
4. Tab 3: Route (2 minutes) — must show . . . 5
5. Tab 4: Live e-Challan (3 minutes) — the showpiece . . . 5
6. Closing (1 minute): under the hood . . . 6
7. If something breaks . . . 6
8. Questions to expect . . . 6

Part B: Learning Guide 7

Lesson 1: Project Overview . . . 7
Lesson 2: System Architecture . . . 8
Lesson 3: Dataset and Feature Engineering . . . 10
Lesson 4: Machine Learning Models . . . 11
Lesson 5: Computer Vision . . . 12
Lesson 6: Live Prediction and Fusion Logic . . . 13
Lesson 7: Streamlit Dashboard . . . 14
Lesson 8: Advanced Features . . . 15
Lesson 9: Testing and Results . . . 16
Lesson 10: Limitations and Future Scope . . . 17
Lesson 11: Evaluating the Computer Vision . . . 18
Lesson 12: Tracking and Speed From Live Video . . . 19
Lesson 13: The Live e-Challan Tab . . . 20
Lesson 14: Time Zone, Speed Limits and Live Video . . . 22
Lesson 15: Route Map, Design Decisions and Bug Stories . . . 22
Complete 2-Minute Presentation Script . . . 23
Common Interview Questions and Short Answers . . . 24
Added Questions and Answers . . . 25
Study Order . . . 26

Part C: Concepts and Talking Points 27

C1. The computer vision concepts used . . . 27

C2. YOLO, CNN, OpenCV and OCR explained 28
C3. How the live feed is captured 30
C4. How the density ratio (frame area) is calculated 30
C5. How vehicle type, color, make and model are found 31
C6. The dataset: what is in it and why it was used 31
C7. How the three ML models work, are trained and are used 32
C7b. The ML part of the project: what the three models represent, and the maths behind them 33
C8. The decimal numbers: what they mean and how they are calculated 35
C9. Why California, and why not India 36
C10. What the project is built around 37
C11. What makes this project different 37
C12. Which cameras to use during the presentation 38
C13. Script to read: the computer vision presentation (about 5 minutes) 38

Part A: Live Demo Walkthrough

You are presenting by running the real app, not slides. Everything below uses the actual tab and button names. Total: about 12 minutes if you do everything, 6 if you only do the **must show** items.

0. Before you present (do this 20 minutes early)

1. **Pick the right time.** The live parts need daylight and traffic in California, so check the clock (California is 12.5 hours behind India until 1 Nov 2026, 13.5 hours after):

California time	India time (until 1 Nov)	Verdict
7-9 AM	7:30-9:30 PM	Best: commute traffic
4-6 PM	4:30-6:30 AM	Best: commute traffic
10 AM-3 PM	10:30 PM-3:30 AM	Fine
8 PM-5 AM	8:30 AM-5:30 PM	Poor: dark, almost no cars

   If your slot is in the poor window, the prediction, grid and route tabs still work; the live monitor will show "calibrating" because there are too few cars. Have a screen recording of it working ready.

2. **Run the readiness check:** `python demo_check.py`. It tests every online service and lists the busiest cameras with working video right now. Note the top one and use it in step 5.
3. **Start the app:** `.venv\Scripts\streamlit run app.py`, open http://localhost:8501.
4. **Warm everything up** by visiting every tab once and pressing Fetch live snapshot, Load grid and Start monitoring (then Stop). The first run of each loads models and is slow; the second is fast.
5. **Have backups:** screenshots of each tab, and a short screen recording of the Live e-Challan tab.

1. Opening (30 seconds, on the header)

Say: "This is a traffic congestion system. It uses **computer vision** to count vehicles on real live California highway cameras, and **machine learning** to predict what traffic should look like from the time of day and weather. It combines the two, and everything you'll see runs on real live data."

Point at the moving cars at the top: "They slow down when the latest reading is congested."

2. Tab 1: Single camera (about 4 minutes) — must show

Camera selection

- Click a dot on the map, or use the District / Filter / Camera boxes. Say: "478 real Caltrans cameras."
- Tick **Watch live video**. Say: "This is the actual live stream, played straight from Caltrans."

Computer vision

- Press **Fetch live snapshot**. Point at the boxes: "YOLOv8 detects each vehicle. I tuned its confidence on 162 frames I labelled myself, which cut the counting error from 7.4 to 4.6 vehicles."
- Point at the **vehicle counter** and **image density ratio**: "Density is how much of the picture the vehicles cover. That becomes the live congestion level."

Machine learning

- Open the **model radio** (LSTM / SVR / Linear Regression) and the **All three models** table. Say: "Three models trained on 48,000 hours of traffic data, with R-squared of 0.977, 0.96 and 0.948."
- Open **Why this prediction?** "Which inputs are unusual right now."

- Point at **Today's forecast**: "The shaded band is the model's typical error on held-out data, so the forecast isn't pretending to be exact."
- Open **When should I leave?** "Suggests a nearby hour with lower predicted congestion."

Live reading vs history (the key idea)

- Point at the caption "Live camera reading: ... historical reading: ..." Say: "The badge is what the camera shows right now. The historical model is context: it is trained on Minnesota traffic, so on a quiet California road it can expect more than is there. I found it made empty roads read High, so the badge follows the camera."
- If the **Unusual congestion detected** banner is visible, explain it: the camera shows far more traffic than history expects, which can mean an accident or event.

Features to tick quickly

- **Use real weather here** (fills conditions from the camera's location).
- **Auto-refresh** (re-fetches on a timer and builds **Recent history**).
- **Download report (PNG)**.
- Mention the **Upload image** and **Snapshot URL** tabs work for any picture.

Optional (30 s): scroll to **Speed check**, press **Check speeds**, then **Identify vehicle color & type**.

3. Tab 2: Monitoring grid (1 minute) — must show

Press **Load grid** (6 cameras). Say: "Each camera gets its own live detection and badge." Point at a "camera unavailable" card: "About 1 in 5 Caltrans cameras serves a 'Temporarily Unavailable' image. My first version read that as an empty road. I found it by testing and fixed it."

4. Tab 3: Route (2 minutes) — must show

Choose **Search places**: type `Burbank Airport` and `Santa Monica Pier`, press **Find places**, pick the matches, press **Check route** (takes about 1 minute; talk while it runs). Say: "It finds the real road path, then every camera within 1.5 km of it, checks them all, and colors the road by each camera's live reading. The overall level is the worst segment." Show **Click the map** (drop two points) and **Pick cameras** in 15 seconds each.

5. Tab 4: Live e-Challan (3 minutes) — the showpiece

1. The camera box is pre-set to **I-5 at Olive** (or pick the camera `demo_check.py` recommended). Say: "Live video here, and the speed limit was filled in automatically from OpenStreetMap."
2. Press **Start monitoring**. Say what you see as it runs:
   - "The detector runs on the video about 4 times a second and tracks each car."
   - "For the first 25 seconds it learns the camera's geometry from the traffic itself. There is no calibration data for these cameras."
   - Then boxes show **estimated speeds**, and the badge and vehicle counter update live.
3. When a car is clearly over the limit a **simulated challan** appears with the car's picture, time, speed, limit and its source. (To force one in a quiet moment, lower the limit and margin first.)
4. Scroll down: **Generate demo challan** (made-up values), the **history** table, **Download history (CSV)**, and **Clear history**.
5. **Say this clearly:** "This is simulated. The speeds are estimates, not validated against radar, so I built in safeguards: it refuses to issue anything if the camera can't be calibrated, a car must stay over the limit for three measurements in a row, and a session issues at most 12 challans."

6. Closing (1 minute): under the hood

- "136 automated tests." (Run `python -m pytest -q` in a terminal if you want to show it.)
- Open `cv_eval/results/report.md`: "The evaluation behind the numbers, including the ideas I tried and rejected: bigger models, sliced detection, a learned classifier."
- Open the README **Limitations and future work**. Say: "The historical model was trained on Minneapolis data, speeds are unvalidated, and the labels are approximate. I'd rather state that than hide it."

7. If something breaks

Problem	What to do
Internet or Caltrans down	Show your screenshots and recording; the README and `cv_eval` results work offline
One camera says "unavailable"	Pick another; it happens to about 1 in 5
Live monitor says "speeds not reliable"	Say "that is the safeguard working", then switch to the recommended camera
Route check is slow	Keep talking; it checks 16 cameras (about 1 minute)
Speed limit says "assumed 65 mph"	OpenStreetMap was slow; it updates by itself when it answers
Something looks frozen	Reload the page and continue

8. Questions to expect

- **Why does the badge follow the camera, not the model?** The model is a Minnesota baseline; taking the worse of the two made empty California roads read High. The camera is the ground truth for "right now".
- **What did you train yourself?** The three prediction models. The detector is pretrained; I tuned and evaluated it.
- **How accurate is the speed?** Honestly, rough: it is calibrated from the traffic itself, not radar.
- **Is the challan real?** No: it is simulated and labelled as such on every card.
- **Why is the historical model trained on Minneapolis data?** It was the best public hourly dataset available; the fix is California sensor data (PeMS), listed as future work.
- **Did you use AI to help?** Answer honestly per your college's rules, and be ready to explain any part.

Part B: Learning Guide

This guide explains the full project in a presentation-friendly way. Use it to learn the project flow, understand the technology stack, and prepare interview answers.

Lesson 1: Project Overview

What the project does

This project predicts road traffic congestion by combining two signals:

1. Historical machine learning prediction
2. Live camera-based computer vision detection

The historical ML side predicts expected traffic volume using time, weather, holiday, and recent traffic-volume features. The computer vision side analyzes a live traffic camera image using YOLO to detect vehicles and estimate how crowded the road looks.

The final result is shown as:

```
Low / Moderate / High / Severe
```

One-line explanation

> *My project predicts traffic congestion by combining historical ML models with live camera-based vehicle detection, and displays the result in a Streamlit dashboard.*

Why this project is useful

Traditional traffic prediction often depends only on historical data. But real roads can suddenly change because of accidents, weather, events, or roadblocks. This project adds a live camera signal so the prediction can react to real-time conditions.

Main project flow

```
Historical traffic data
        |
        v
Feature engineering
        |
        v
Train ML models
        |
        v
Predict expected traffic volume


Live traffic camera image
        |
        v
YOLO vehicle detection
        |
        v
Estimate live congestion


ML result + CV result
        |
        v
Fusion logic
        |
        v
Final congestion prediction
        |
        v
Streamlit dashboard
```

Presentation paragraph to memorize

> *This project predicts traffic congestion using two independent signals. First, historical traffic data is used to train ML models that predict traffic volume based on time, weather, and lag features. Second, a live camera image is analyzed using YOLO to detect vehicles and estimate image density. These two outputs are fused by choosing the more severe congestion level, because in a warning system under-reporting visible congestion is worse than over-reporting it. The final result is displayed in a Streamlit dashboard.*

Lesson 2: System Architecture

Four main layers

```
1. Data Layer
   Historical CSV, live camera feed, weather API

2. Model Layer
   Linear Regression, SVR, LSTM, YOLO

3. Logic Layer
   Feature engineering, live prediction, fusion, anomaly detection

4. UI Layer
   Streamlit dashboard
```

Main files and responsibilities

File	Role
`src/data_loader.py`	Loads CSV and creates model features
`src/train_regression.py`	Trains Linear Regression and SVR
`src/train_dl.py`	Trains LSTM
`src/evaluate.py`	Trains all three models and compares them
`src/cv_module.py`	Runs YOLO vehicle detection
`src/live_predict.py`	Builds live features, predicts volume, and fuses labels
`src/live_cameras.py`	Fetches live Caltrans camera data
`src/weather.py`	Fetches current weather from Open-Meteo
`src/speed_estimation.py`	Estimates vehicle speed from live video
`src/anpr.py`	Attempts license plate OCR
`src/vehicle_attributes.py`	Estimates vehicle color and body type
`src/echallan.py`	Generates simulated challan summaries
`src/prediction_log.py`	Stores local prediction history
`src/report.py`	Builds downloadable PNG reports
`src/camera_time.py`	California time for model inputs (the cameras' clock)
`src/speed_limits.py`	Posted speed limit for a camera's road from OpenStreetMap
`src/vehicle_tracking.py`	Vehicle tracking, camera calibration and speed estimation
`src/live_monitor.py`	Background live monitor: detection, speed, automatic challans
`src/live_video.py`	Plays a camera's live video stream in the page
`src/route_map.py`	Road path, place search, cameras along a route
`src/ui_theme.py`	The control-room look: header, badges, counter
`cv_eval/`	Labelled frames and the vision evaluation (Lesson 11)
`app.py`	Streamlit dashboard connecting everything

Runtime flow

When the user selects a live camera:

1. `app.py` fetches a live camera frame.
2. `cv_module.py` detects vehicles using YOLO.
3. `live_predict.py` builds a feature row using current time, weather, and lag estimates.
4. Saved ML models predict traffic volume.
5. Predicted volume is converted into a congestion label.
6. CV density is converted into a congestion label.
7. The two labels are fused by taking the more severe one.
8. The final result appears in the Streamlit dashboard.

Interview answer

> *The project is modular. The data pipeline is handled in `data_loader.py`, model training is split between `train_regression.py` for Linear Regression and SVR and `train_dl.py` for LSTM, computer vision is handled in `cv_module.py`, and live inference plus fusion is handled in `live_predict.py`. Finally, `app.py` connects everything into a Streamlit dashboard.*

Lesson 3: Dataset and Feature Engineering

Dataset

The project uses the Metro Interstate Traffic Volume dataset. It contains hourly traffic-volume readings along with weather and time-related information.

The target variable is:

```
traffic_volume
```

The model tries to predict this value.

Important features

The project creates features such as:

Feature Type	Examples	Why it matters
Time features	hour, day of week, month	Traffic depends strongly on time
Cyclical features	`hour_sin`, `hour_cos`	Helps model understand that 23:00 and 00:00 are close
Weather features	temperature, rain, snow, clouds	Weather can affect traffic
Holiday feature	`is_holiday`	Holidays change travel behavior
Lag features	`lag_1h`, `lag_3h`, `lag_24h`	Recent traffic is very useful for prediction
Rolling feature	`rolling_mean_3h`	Smooths recent traffic trend

Why sin/cos time features are used

Raw hour values can mislead a model. For example, hour 23 and hour 0 are numerically far apart, but in real time they are next to each other.

Sin/cos encoding fixes this by representing time as a cycle.

Why lag features matter

Traffic is time-dependent. If traffic was high one hour ago, it is likely still high now. Lag features give the model recent history.

This is one of the biggest reasons the Linear Regression and SVR models became accurate.

Chronological split

The project uses chronological train/validation/test splitting instead of random splitting.

This matters because traffic data is time-series data. If future rows are mixed into training, the model can accidentally learn from the future. That is called data leakage.

Interview answer

> *I used cyclical time features because time repeats daily and weekly. I also added lag and rolling features because traffic volume is strongly dependent on recent traffic. Since this is time-series data, I used chronological splitting instead of random splitting to avoid data leakage from future records.*

Lesson 4: Machine Learning Models

The project trains three models:

```
Linear Regression
SVR
LSTM
```

Linear Regression

Linear Regression is used as a baseline model. It is simple, fast, and explainable.

It became much stronger after adding:

1. Sin/cos cyclical time features
2. Lag and rolling traffic features

SVR

SVR means Support Vector Regression. The project uses an RBF kernel, which can model nonlinear relationships better than simple Linear Regression.

SVR is slower to train on large datasets, so it is trained on an 8,000-row subsample for speed.

Hyperparameters were tuned separately using `RandomizedSearchCV` with `TimeSeriesSplit`.

LSTM

LSTM is a deep learning model used for sequence/time-series prediction.

In this project, it looks at a 24-hour window of previous features to predict traffic volume.

The LSTM performs best, but the gain over SVR is not huge because the lag features already give the classical models strong time-history information.

Metrics

The main evaluation metrics are:

Metric	Meaning
MAE	Average absolute error
RMSE	Penalizes larger errors more strongly
R2	How much variance the model explains

Metrics are calculated in:

```
src/train_regression.py
src/train_dl.py
```

`src/evaluate.py` trains all three models, prints the comparison, and creates the comparison plot using Matplotlib.

Interview answer

> *I trained Linear Regression as a baseline, SVR with an RBF kernel for better nonlinear modeling, and an LSTM for time-series sequence learning. The LSTM gave the best result, but SVR was also strong because lag features already captured recent traffic behavior.*

Lesson 5: Computer Vision

Purpose

The computer vision module gives the project a live real-world signal.

Instead of relying only on historical data, the project looks at a current traffic camera frame and detects vehicles.

YOLO

YOLO is used for object detection. It detects vehicle classes such as:

```
car
motorcycle
bus
truck
```

The project uses YOLOv8s with a detection confidence of 0.08, tuned on labelled frames (Lesson 11).

OpenCV

OpenCV is used for:

1. Loading images
2. Reading video frames
3. Handling image arrays
4. Drawing annotations

Density calculation

After YOLO detects vehicles, the project calculates how much of the image is covered by vehicle bounding boxes.

```
density_ratio = total vehicle bounding-box area / frame area
```

Then density is mapped to:

```
Low / Moderate / High / Severe
```

Limitation

YOLO can struggle on low-resolution, nighttime, or blurry traffic camera images. This is a real-world limitation and should be mentioned honestly.

Interview answer

The CV module uses YOLOv8s to detect vehicles in a traffic camera frame. OpenCV loads the image, YOLO finds vehicle bounding boxes, and the system calculates an image-density ratio from the total vehicle area. That density is mapped to a congestion label.

Lesson 6: Live Prediction and Fusion Logic

Why fusion is needed

The historical ML model and live camera analysis give different kinds of information.

Historical model:

```
What traffic is expected to be like at this time
```

Live camera:

```
What the road looks like right now
```

Both are useful, so the project combines them.

Fusion strategy

The final label is the more severe of the two labels:

```
final_label = max_severity(historical_label, cv_label)
```

Example:

Historical ML	Live CV	Final
Low	Severe	Severe
High	Moderate	High
Moderate	Moderate	Moderate

Why not weighted average?

A weighted blend was tried first. But it failed in an important case: a visibly gridlocked road was detected as Severe by CV, but the historical model predicted Low, and the weighted blend still reported Low.

That is dangerous for a warning system.

So the project chooses the more severe signal to avoid under-reporting visible congestion.

Anomaly detection

If the historical model and live camera disagree by two or more severity levels, the project flags it as unusual congestion.

This can indicate:

1. Accident
2. Road closure
3. Event traffic
4. Sudden congestion not present in historical data

Interview answer

> *The fusion logic takes the more severe of the historical ML label and the live CV label. I chose this because a previous weighted blend under-reported a visibly jammed image as Low. For a warning system, missing real congestion is worse than being slightly conservative.*

Lesson 7: Streamlit Dashboard

Purpose

The Streamlit app is the user interface. It lets a user:

1. Pick a live Caltrans camera
2. Upload an image
3. Paste a snapshot URL
4. Set weather conditions
5. Choose a model
6. View final congestion prediction
7. See detected vehicles
8. View forecast chart
9. Download report
10. Check route or multiple cameras

Important Streamlit concepts

Concept	How it appears
Widgets	sliders, buttons, select boxes, checkboxes
Tabs	Single camera, Monitoring grid, Route, Live e-Challan
Session state	remembers selected camera, current frame, speed results
Reruns	Streamlit reruns the script after user interaction
Fragments	used for auto-refresh without blocking the whole app
Cache	avoids rerunning expensive detection/model prediction unnecessarily

Important bug story

Streamlit tabs still execute their code even when the tab is not visually active. This caused stale inputs from another tab to overwrite the current camera frame.

The project fixes this by tracking which source acted most recently:

```
frame_source = live / upload / url
```

Auto-refresh bug story

An early auto-refresh design used `sleep` and rerun. That blocked the app and made the UI unstable.

The project now uses Streamlit fragments and checks whether enough time has passed before fetching again.

Interview answer

> *Streamlit reruns the script whenever a widget changes, so I used session state to preserve selected cameras, frames, speed results, and source tracking. I also used caching for expensive YOLO/model calls and fragments for safe auto-refresh.*

Lesson 8: Advanced Features

Live cameras

`src/live_cameras.py` fetches real public Caltrans CCTV camera data.

The dashboard lets the user select district, route/location, and camera.

Real weather

`src/weather.py` fetches current weather from Open-Meteo using the camera's latitude and longitude.

This can auto-fill:

```
temperature
rain
snow
cloud cover
weather type
```

Forecast chart

The app predicts traffic for every hour of the day and shows it with Altair.

It also displays an error band using each model's typical MAE, so the forecast does not look falsely exact.

Pydeck map

The app uses pydeck for the camera map.

Important bug story:

Per-row accessors for color/radius broke click selection on the map. The fix was to separate the map into layers:

1. Non-pickable congestion-color layer
2. Pickable camera layer
3. Selected-camera highlight layer

Route tab

The route tab lets users pick cameras, search places, or click the map. It draws the real road path colored by nearby cameras (Lesson 15). The final route congestion is the worst segment.

This makes sense because one jammed segment can affect the whole route.

Speed estimation

`src/speed_estimation.py` estimates vehicle speeds from live HLS video streams.

It uses:

1. YOLO detections across multiple frames

2. Nearest-centroid tracking
3. Stream timestamps
4. Lane-width calibration

Important detail:

It uses the stream's internal timestamp, not wall-clock time, because HLS video can download buffered frames faster than real time.

This is the older speed check in the Single camera tab. The newer Live e-Challan tab fits a camera model from the traffic itself (Lesson 12).

ANPR / OCR

`src/anpr.py` uses EasyOCR to attempt license plate recognition from a selected vehicle crop.

Limitation:

Real traffic camera footage often has blurry, angled, hidden, or unreadable plates.

Vehicle attributes

`src/vehicle_attributes.py` estimates:

1. Vehicle color using image color clustering
2. Vehicle body type using CLIP zero-shot classification

This is best-effort and not guaranteed.

Simulated e-challan

`src/echallan.py` generates an illustrative fine summary for a speeding vehicle.

It is a demo only:

```
No real fine is issued.
No government system is connected.
No payment is charged.
```

JSON logs

The project stores local logs for:

1. Prediction history
2. Simulated challan history

These are local JSON files and are not sent anywhere.

Lesson 9: Testing and Results

Test suite

The project has pytest tests covering:

1. Data loading and feature engineering
2. Fusion logic
3. Live prediction
4. Weather mapping
5. Report generation

6. Prediction logs
7. Speed estimation math
8. OCR behavior
9. E-challan logic
10. Vehicle attributes

The test suite passed:

```
136 passed
```

Why tests matter

Tests prove that the important logic works and that future changes do not break existing behavior.

Strong presentation point

> *I added tests not only for the basic model pipeline, but also for edge cases like fusion behavior, report generation, weather mapping, OCR fallback, speed estimation math, and local logs.*

Interview answer

> *I used pytest to test the core behavior of the project. The tests cover data processing, live prediction, fusion logic, reports, weather, speed estimation, OCR, challan generation, and vehicle attributes. This helps make the project more reliable and easier to change.*

Lesson 10: Limitations and Future Scope

Main limitations

1. The historical model is trained on Minneapolis-area data but the live cameras are in California.
2. YOLO can struggle with low-resolution or nighttime images.
3. Speed estimates are not validated against radar (Lesson 12); the older speed check needs manual lane-width calibration.
4. OCR is often unreliable on real traffic camera footage.
5. The system is a demonstration project, not a government-certified traffic enforcement system.
6. The fusion rule is conservative, but not learned from paired ground-truth camera-volume data.

Why the dataset mismatch matters

The historical model learns traffic patterns from one dataset, while live cameras come from Caltrans. Without paired camera + sensor data for the exact same locations, the model cannot be perfectly calibrated to each camera.

Future improvements

1. Train on paired camera and sensor data from the same road locations.
2. Fine-tune YOLO on traffic-camera images, especially nighttime/low-resolution footage.
3. Add a stronger multi-object tracker for speed estimation.
4. Add camera calibration for more accurate speed measurement.
5. Improve OCR with better plate-specific preprocessing.

6. Deploy the app publicly with a reliable backend.
7. Add live incident/event feeds.

Interview answer

> *The main limitation is that the historical dataset and live camera feeds are not perfectly paired by location, so the model is best understood as a demonstration of an end-to-end architecture. Accuracy could improve with paired traffic sensor and camera data, YOLO fine-tuning on local traffic cameras, and proper camera calibration for speed estimation.*

Lesson 11: Evaluating the Computer Vision

Why this lesson exists

The R-squared of 0.977 belongs to the historical model only. For a long time the computer vision part had no accuracy number at all. This lesson is the measurement of the vision part, found in `cv_eval/`.

The evaluation set

Item	Value
Frames	162 real Caltrans frames, all 12 districts, captured 2026-10-13 around 09:00-09:25 PDT
Extra frames	Interstate and US-route cameras from the busy districts, so there are congested examples
Labels per frame	vehicle count, congestion class (Low / Moderate / High / Severe), usable yes/no
Usable	124 frames (Low 92, Moderate 22, High 8, Severe 2)
Unusable	33 "Temporarily Unavailable" images, 3 glare, 1 haze, 1 parking lot

Honest note about the labels: they were made by looking at the frames (with Claude's help), not taken from a sensor. Counts are approximate (small distant cars are counted, which makes the detector look worse) and the classes are a visual judgement. Only 10 frames are High or Severe, so results for those classes are weak evidence. Re-check them with `python cv_eval/make_sheets.py --labels`.

Finding 1: one in five cameras was "Temporarily Unavailable"

33 of 162 sampled cameras served a mostly white placeholder image. A detector finds no cars in it, so the app reported an empty road ("Low"). The placeholder is about 86 percent near-white pixels, and the most washed-out real frame was 22 percent, so a 60 percent cutoff separates them. `analyze_frame` now raises `CameraUnavailableError`, and the UI shows "camera unavailable" instead of a badge.

Finding 2: the detector undercounted, and a lower confidence fixed most of it

Setting (YOLOv8s, 640 px)	Count error (MAE)	Average miss	Within tolerance
Confidence 0.25 (YOLO default)	7.43 vehicles	-7.3 (too low)	39 percent
Confidence 0.08 (shipped)	4.55 vehicles	-2.5	59 percent

On dense frames (21 or more vehicles) the error fell from 23.5 to 12.6; on near-empty frames it moved from 1.55 to 1.41, so empty roads did not get worse. The app now uses `DEFAULT_CONF = 0.08`.

Finding 3: what did not help

- **A bigger model:** YOLOv8m at 0.08 reached 4.15 (a small gain) but takes 0.56 seconds per frame against 0.24, which hurts when a route checks 16 cameras. The app stays on YOLOv8s.
- **Higher input size (1280 px):** no gain.
- **Refitting the density thresholds** (0.05 / 0.15 / 0.30) on labelled data with cross-validation: worse macro-F1 than the original thresholds once the detector was fixed.
- **Sliced inference** (also searching four overlapping windows so far cars have more pixels): 4.55 to 4.32, only about 5 percent, at roughly five times the compute. Kept as an option, not the default.
- **A learned congestion classifier** (logistic regression, random forest, gradient boosting): balanced accuracy 0.41-0.54 against 0.62 for the plain thresholds. 124 frames is too few for a learned model to win.

Reading the label metrics correctly

74 percent of usable frames are Low, so "always answer Low" already scores 0.74 accuracy. That is why the report also gives **balanced accuracy** (average recall per class) and **macro-F1**.

Setting	Accuracy	Balanced accuracy	Macro-F1
Before tuning (confidence 0.25)	0.73	0.32	0.33
Shipped (confidence 0.08)	0.75	0.62	0.64
Always answer "Low"	0.74	0.25	-

Caveats to say out loud

The confidence value was picked from a handful of options on the same frames it is scored on, so the gain is slightly optimistic. The set is daytime only, from one 25-minute window. Dense traffic is still undercounted (one wide freeway frame with about 45 vehicles was read as 9). Nothing was fine-tuned: that needs bounding boxes, not the counts collected here.

Interview answer

> *I measured the vision part on 162 live frames I labelled across all 12 districts. That found two real problems: a fifth of the cameras serve an unavailable image that looked like an empty road, and the detector at its default confidence undercounted by about 7 vehicles. Lowering the confidence cut the counting error from 7.4 to 4.6. I also tried bigger models, sliced inference, and a learned classifier, and kept the simple version because they did not clearly help. The labels are approximate and daytime only, so I treat the numbers as evidence, not proof.*

Lesson 12: Tracking and Speed From Live Video

The problem

Speed is distance over time. The time is easy (the video stream has a clock). The distance is hard: the cameras publish no calibration, so a pixel on the screen has no known size in metres.

The pipeline (src/live_monitor.py and src/vehicle_tracking.py)

1. **Frames:** the stream is 15 frames per second. Every 4th frame is analysed (about 3.75 per second) and its time is read from the stream's own clock, because network delays make wall-clock time wrong.
2. **Detect and track:** YOLO finds the vehicles. A simple tracker predicts where each vehicle should be next from its recent motion and matches the nearest detection, so the same car keeps one id.

3. **Fit a camera model from the cars themselves** (first about 25 seconds). For a camera looking along a flat road, a car's bottom edge in the image (y) and its pixel width (w) follow a line: `y = horizon + a * w`. Fitting that line over many cars gives the horizon row and the camera height (`a * 1.8 m`, taking a typical car to be 1.8 m wide). If the fit is not convincing (too few cars, no trend, implausible height), speeds are switched off for that camera.
4. **Pixels to metres:** the focal length is assumed from a typical 62 degree field of view. Then `distance ahead = f * height / (y - horizon)` and `sideways = (x - centre) * distance / f`.
5. **Speed:** for each car, take its last 2.5 seconds, fit a straight line to its ground position over time, and take the slope. Only cars between 8 and 75 metres from the camera count.
6. **One correction:** the zoom was only assumed, so speeds are scaled so the median tracked car matches the "typical traffic speed" setting (90 percent of the posted limit by default). If that needs a correction outside 0.4 to 2.5 times, the camera is marked unreliable and issues nothing.
7. **Flag:** the car must stay over the limit plus a margin for 3 measurements in a row.

The older method (Single camera, Speed check)

You measure a lane's width in pixels and type it in. Speed is pixels moved times (3.7 m divided by lane pixels) divided by time. It ignores perspective and depends on your measurement.

How far to trust it

- **Verified:** a synthetic car moving at a known speed reads within 4 percent.
- **Not verified:** there is no radar or other ground truth. On the real cameras tried, the raw model needed a correction of about 2 times, and far-away cars read much faster than near ones in the same traffic. That is why the distance band and the traffic-speed scaling exist.
- **What the numbers mean:** treat speeds as "faster or slower than the rest of the traffic", not exact mph.

Interview answer

> *There is no calibration data for these cameras, so I fit a flat-road camera model from the traffic itself: a car's ground row and width follow a line that gives the horizon and camera height. With an assumed focal length I convert image positions to road distance, track each car over a couple of seconds, and take the slope as speed. Because the zoom is unknown I scale speeds to a typical traffic speed, and if the model can't be calibrated the app refuses to issue anything. My math tests pass on synthetic cars within 4 percent, but I haven't validated against radar, so I describe the speeds as estimates.*

Lesson 13: The Live e-Challan Tab

What you see

One tab with, top to bottom: camera selection (district, filter, camera), the camera's live video, the automatic speed limit, a Start monitoring button, the annotated live detection (boxes with estimated speeds, a vehicle counter, a congestion badge), the challans issued for real cars, then a form to make a challan from made-up values, the challan history, a CSV download and a clear-history button.

How it works

- **Live video:** an hls.js player runs in your browser and plays Caltrans's stream directly (their server allows this). Nothing is proxied or recorded by the app.
- **The monitor:** a background thread (`LiveMonitor`) reads the stream and runs detection, tracking and speed. It uses its **own** YOLO instance because the app's shared model is not safe to use from two threads. The page polls it once a second with a Streamlit fragment, and the monitor stops itself after the run time or if the page stops asking (45 seconds).
- **A challan:** when a car is clearly over the limit, the monitor crops its picture, optionally reads the plate and identifies its color and type, and builds a simulated challan. Plate reading and identification run in a separate worker so the video does not stall.
- **What a challan shows:** reference number, time in California time with its UTC offset, location, vehicle (with a note if no plate could be read), appearance, estimated speed, speed limit and where it came from, the illustrative fine in rupees, and the simulation disclaimer.
- **Privacy:** evidence pictures stay in memory only. The on-disk history holds text fields only.

Safeguards

speeds far above the surrounding traffic (over 1.6 times the typical speed, or over 100 mph) are treated as tracking errors and never flagged (found in a live test: half of the first 12 challans showed 97 to 119 mph, which were tracks jumping between cars; after the fix they were 60 to 79 mph); 3 confirmations in a row; at most 12 challans per session, at least 3 seconds apart; no challans at all if the camera could not be calibrated; a warning if the limit is set below typical traffic speed (which would flag most cars).

Plate, color, make and model

- **Plate (EasyOCR):** often unreadable on real footage; the challan then says "NOT READ".
- **Color and body type:** CLIP zero-shot and color clustering; best-effort guesses.
- **Make and model:** tried and gated. The median detected car is only 19 pixels wide, and even the 20 largest crops gave 5-15 percent confidence, so crops under 70 px are refused and a model is reported only above 30 percent confidence. On these cameras it will rarely say anything, on purpose.

Bug stories (good for interviews)

- **Hidden feature:** the e-challan used to appear only after a speed check found a speeder, so nobody saw it. It now has its own tab.
- **Runaway challans:** a test with the limit below normal traffic speed produced 293 challans in two minutes. That led to the cap, the spacing, and the warning.
- **Wrong wording:** a challan with no plate said "entered manually". It now says no plate could be read.

Interview answer

> *The Live e-Challan tab watches a real camera, tracks vehicles, and generates a clearly-labelled simulated challan from a real observed vehicle, with its picture, estimated speed, time, location and the posted limit. It runs in a background thread with its own detector, has safeguards against false or runaway challans, and refuses to issue anything when the camera can't be calibrated. It is a demonstration of the pipeline, not enforcement.*

Lesson 14: Time Zone, Speed Limits and Live Video

The time-zone bug

The historical models take an hour, weekday and month. The app used the computer's own clock. A laptop on India time (16:17) asked the models about Sunday afternoon rush hour while it was 03:46 in California, and the app showed a false "Unusual congestion detected". Found by a full test pass. Fix: `src/camera_time.py` gives California time (`America/Los_Angeles`), used for predictions, the forecast, the report footer and challan timestamps.

The automatic speed limit (src/speed_limits.py)

OpenStreetMap stores a `maxspeed` tag on many roads (Overpass API, free, no key). The app finds the road the camera watches and reads it:

1. A nearby road whose route reference matches the camera's (so a camera on an overpass reports the freeway, not the street crossing it), nearest first.
2. Otherwise the nearest motorway, then the nearest road of any kind.
3. If the road has no tag, a typical value for its class is used and labelled "assumed".

Checked live: the I-110 Arroyo Seco Parkway reports 55 mph and the I-5 freeway 65 mph, matching the posted limits.

Why it never blocks the page: the public Overpass servers are often slow or busy. The lookup runs in a background thread, the page shows a clearly-labelled provisional value (65 mph for Interstate and US routes) and updates itself when the real answer arrives. It tries three mirrors, and a found limit is cached on disk so each camera needs only one successful lookup. Limits are only as good as what OpenStreetMap contributors entered.

Live video in the browser

An hls.js player (version pinned) plays the camera's HLS playlist. About two thirds of cameras have a stream, and many of those are offline at any moment, so a dead stream shows a message instead of a blank box.

Interview answer

> *I get the posted speed limit from OpenStreetMap by finding the road the camera is on and reading its maxspeed tag. Because the public server is slow, I do it in a background thread with a provisional value, mirrors, retries and a disk cache. I also found and fixed a bug where the models used the laptop's clock instead of California's.*

Lesson 15: Route Map, Design Decisions and Bug Stories

The Route tab

Three ways to build a route, all giving the same result:

Mode	How
Pick cameras	choose 2 or more cameras in a district
Search places	type a start and destination (Nominatim place search, one request per second)
Click the map	click once for the start, again for the destination

The app gets the real road path from OSRM (OpenStreetMap routing, free), finds every camera within 1.5 km of it (up to 16, evenly spaced), checks each one, and colors the road by the nearest camera: solid within 2 km, faded 2-8 km away (a weaker guess), gray beyond 8 km. The overall route level is the worst segment. Colors are estimates from scattered snapshots, not continuous traffic data like a navigation app.

Design decisions worth mentioning

- **Per-segment live reading:** each route segment and each grid badge shows that camera's own live reading. The historical label is one value for the whole time and weather, so putting it into every segment put a "Moderate" floor under all of them. It is applied once, to the overall reading.
- **Free services only:** Caltrans, OSRM, Nominatim, Open-Meteo and OpenStreetMap need no API key.
- **A dark control-room theme** with moving cars that slow down when traffic is congested, glowing badges, and an animated counter, all in CSS.

Bug stories (a good list for questions)

Bug	What happened	Fix
Stale selectbox label	Changing a filter crashed with a TypeError	validate the stored value and reset
Auto-refresh loop	A Streamlit fragment also runs on every full rerun, causing an endless refresh	elapsed-time guard
Everything "Moderate"	the shared historical label was fused into every camera	per-camera live label, fuse once
Placeholder as empty road	"Temporarily Unavailable" images read as Low	detect and refuse them
Wrong clock	models used the laptop's time zone	California time
Runaway challans	293 in one test	cap, spacing, warning
Cropped map	a tall route was cut off in a short wide map	a proper fit-to-view calculation

What is distinctive (for "what makes it unique")

Not any single piece, but the combination: a historical model and live computer vision fused with a stated reason; real, messy live data with real failures found and fixed; measuring the vision part and keeping only what helped; and going beyond a dashboard to routes, tracking, speed, and a simulated enforcement pipeline, all with the limits written down.

Complete 2-Minute Presentation Script

My project is called Smart Traffic Congestion Prediction. The goal is to estimate road congestion by combining historical machine learning with live computer vision.

The historical side uses the Metro Interstate Traffic Volume dataset. I first clean the data and create features such as cyclical time features, weather features, holiday information, and lag features like traffic volume from one, three, and twenty-four hours ago. These features are used to train three models: Linear Regression, SVR, and LSTM.

Linear Regression acts as a simple baseline, SVR captures nonlinear patterns, and LSTM handles time-series sequences using a 24-hour window. The models are evaluated using MAE, RMSE, and R2.

The live side of the project uses YOLOv8s to detect vehicles from a traffic camera frame. It counts cars, buses, trucks, and motorcycles, then estimates image density based on vehicle bounding-box area. This gives a live camera-based congestion label.

The final prediction combines the historical ML label and live CV label using a conservative fusion rule: it chooses the more severe congestion level. I used this because a weighted blend can under-report visible congestion, which is risky for a warning system.

The full system is displayed in a Streamlit dashboard. Users can select live Caltrans cameras, upload images, use real weather, view forecasts, check routes, download reports, and run experimental features like speed estimation, OCR, and simulated challan generation.

Beyond the prediction, a Live e-Challan tab runs detection and tracking on the live video, estimates speeds, and generates clearly labelled simulated challans from real observed cars. I measured the vision part on 162 labelled frames, which cut the counting error from 7.4 to 4.6 vehicles, and I documented what did not work.

The project is modular, with separate files for data loading, model training, computer vision, live prediction, weather, reports, and UI. I also added a pytest suite covering the core logic and edge cases. The main limitation is that the historical dataset and live camera locations are not perfectly paired, so future work would involve training on paired camera-sensor data and fine-tuning YOLO for traffic-camera footage.

Common Interview Questions and Short Answers

Q1. What problem does your project solve?

It predicts road congestion using both historical traffic patterns and live camera conditions.

Q2. Why did you use both ML and computer vision?

Historical ML predicts expected traffic, while computer vision captures what is happening right now. Combining them makes the system more responsive.

Q3. Why use lag features?

Traffic is strongly dependent on recent traffic. Lag features give the model memory of recent conditions.

Q4. Why use sin/cos features?

Time is cyclical. Sin/cos encoding helps the model understand that hour 23 and hour 0 are close.

Q5. Why chronological split?

Because this is time-series data. Random splitting could leak future information into training.

Q6. Why use three models?

Linear Regression gives a baseline, SVR captures nonlinear patterns, and LSTM models sequences.

Q7. Why is the LSTM improvement over SVR not huge?

Because lag and rolling features already give SVR strong recent-history information.

Q8. How does YOLO help?

YOLO detects vehicles in live camera frames, allowing the system to estimate real-time congestion visually.

Q9. Why take the more severe fusion label?

Because under-reporting visible congestion is worse than being conservative in a warning system.

Q10. What are the main limitations?

Dataset-camera mismatch, low-resolution camera images, manual speed calibration, and unreliable OCR on real footage.

Q11. What would you improve next?

Use paired camera-sensor data, fine-tune YOLO on traffic footage, add better tracking, and deploy the app.

Added Questions and Answers

Q12. How is speed calculated?

By tracking each car across frames and converting image position to road distance with a camera model fitted from the traffic itself, then scaling to a typical traffic speed. It is an estimate, not a measurement.

Q13. How accurate is the speed?

Honestly, rough. The math is tested on synthetic cars (within 4 percent) but it is not validated against radar, so I describe it as "faster or slower than the rest of the traffic".

Q14. Is the challan real?

No. Every challan is simulated and labelled as such. Nothing connects to any authority or registry.

Q15. Why is the historical model trained on Minneapolis data?

It was the best public hourly dataset with weather and traffic volume that I could use. It does not match California, so its output is a time-of-day and weather baseline. The fix is California sensor data (PeMS).

Q16. How did you evaluate the vision part?

I labelled 162 live frames from all 12 districts and measured counting error and label accuracy. Tuning the confidence cut the error from 7.4 to 4.6 vehicles. I also tried bigger models, sliced inference and a learned classifier and kept the simple version.

Q17. What did you train yourself?

The three prediction models (Linear Regression, SVR, LSTM). The detector is pretrained; I tuned and evaluated it.

Q18. Can it identify the car's make and model?

Mostly no. The median car on these cameras is 19 pixels wide, so the app only reports a make or model when the crop is big enough and the model is confident, which is rare.

Q19. How does the app get the speed limit?

From OpenStreetMap, by finding the road the camera watches. It runs in the background with a fallback if the service is slow.

Q20. What are the biggest limitations?

The historical model's data mismatch, unvalidated speeds, approximate labels from a small daytime set, and cameras that are offline or have a sideways view.

Study Order

Use this order while learning:

1. `README.md`
2. `src/data_loader.py`
3. `src/train_regression.py`
4. `src/train_dl.py`
5. `src/evaluate.py`
6. `src/cv_module.py`
7. `src/live_predict.py`
8. `src/live_cameras.py`
9. `src/weather.py`
10. `src/speed_estimation.py`
11. `src/anpr.py`
12. `src/vehicle_attributes.py`
13. `src/echallan.py`
14. `src/prediction_log.py`
15. `src/report.py`
16. `app.py`
17. `tests/`
18. `src/vehicle_tracking.py`
19. `src/live_monitor.py`
20. `src/speed_limits.py`
21. `src/camera_time.py`
22. `src/route_map.py`
23. `cv_eval/evaluate_cv.py`

Part C: Concepts and Talking Points

Short explanations you can use when someone asks "what is X?" or "how does Y work?", plus two scripts to read aloud. Each section is written so you can say it in your own words.

C1. The computer vision concepts used

Concept	Where it is used	In plain words
Object detection (YOLOv8)	Counting vehicles in every frame	A neural network that finds objects in an image and draws a box around each with a class (car, truck, bus, motorcycle) and a confidence score.
Confidence thresholding	The 0.08 setting you tuned	Keep a detection only if the model is at least this sure. A lower value finds more cars but risks false ones.
Non-maximum suppression and merging boxes	Inside YOLO, and in the sliced-detection option	Removes duplicate boxes on the same car. Sliced detection also merges boxes found in overlapping tiles, using intersection over the smaller box.
Multi-scale (sliced) inference	The tiled option, which was measured and rejected	Run the detector on zoomed windows so tiny, distant cars get more pixels.
Density estimation	The congestion level	Vehicle-box area divided by image area, turned into Low to Severe.
Image quality validation	The "camera unavailable" check	Measure pixel statistics (about 86 percent near-white) to spot a placeholder image instead of a real scene.
Video decoding and timestamps	Live monitor, speed check	Reading frames from a live HLS stream and using the stream's own clock for timing.
Object tracking (data association)	Live monitor	Matching the same car from frame to frame: predict its next position from its motion, then match the nearest detection.
Camera geometry (pinhole model, horizon, ground plane)	Speed estimation	Treat the road as flat and use where a car's bottom edge sits in the image to work out how far away it is.
Perspective and scale	Speed, the distance band	Far cars cover few pixels, so tiny pixel errors become large speed errors. That is why only cars 8 to 75 metres away count.
Motion estimation	Speed	The slope of a car's position over time.
Zero-shot classification (CLIP)	Vehicle body type, make and model	A model that matches an image to text prompts like "a sedan", with no training on traffic data.
Color analysis (k-means, HSV)	Vehicle color	Cluster the car's pixels, ignore shadows and glare, and name the dominant color.

Concept	Where it is used	In plain words
OCR (EasyOCR)	Plate reading	Find and read text in a cropped image. It usually fails here because the plates are tiny or hidden.
Image preprocessing	Plate and crop handling	Cropping, upscaling, and drawing boxes and labels on frames (OpenCV).
Evaluation of a vision system	`cv_eval/`	Hand-labelled frames, counting error (MAE), a confusion matrix, balanced accuracy and cross-validation.

What is pretrained and what is yours. YOLO, CLIP and EasyOCR are pretrained models you use. Your own work is everything around them: the placeholder check, the density-to-label rule, the tracker, the camera-geometry and speed estimation, the evaluation, and the tuning.

C2. YOLO, CNN, OpenCV and OCR explained

What is YOLO

YOLO stands for "You Only Look Once". It is a neural network that finds objects in an image. You give it a picture and it returns a list of detections: a **box**, a **class** (car, truck, bus...) and a **confidence score** from 0 to 1 (the `car 0.66` on the boxes in the app).

It is called "You Only Look Once" because older methods scanned an image over and over, window by window, which was slow. YOLO looks at the whole image in one pass and predicts every box at once, which makes it fast enough for live video.

How it works in four steps: the image is resized and passed through a convolutional neural network; the network predicts many candidate boxes, each with a class and a confidence; non-maximum suppression removes duplicate boxes on the same object; boxes below your confidence threshold (0.08) are dropped.

Your version is **YOLOv8s** ("s" means small), pretrained on COCO, a dataset of everyday objects. You did not train it. You tuned it and evaluated it on frames you labelled. It takes about 0.24 seconds per frame on your CPU.

Say: "YOLO is a pretrained object detector that finds vehicles in one fast pass over the image. I use it to count cars and measure how crowded the road is, and I tuned its confidence threshold on frames I labelled."

How YOLO detects the shape of a car

YOLO does not have a hand-written rule like "a car has four wheels and a rectangle body". It learns what cars look like from millions of labelled examples. During training it made guesses, compared them with human-drawn boxes, and adjusted its internal numbers (weights) until the guesses matched. The "shape" of a car is a learned combination of parts and textures, not a stored outline:

```
Early layers   -> edges and corners, light and dark patches
Middle layers  -> curves, wheels, windows, headlights, textures
Deep layers    -> "this whole arrangement looks like a car"
```

It struggles when a car lacks those cues: very small or far cars, blur, night, glare, or cars hidden behind others. That is why the app undercounts dense, distant traffic.

What is a CNN

A CNN (convolutional neural network) is a neural network designed for images. It keeps the picture's 2D layout and looks for small local patterns.

1. **Convolution:** a tiny grid of numbers called a **filter** (for example 3 by 3 pixels) slides across the whole image and measures how strongly each spot matches a pattern (an edge, a curve, a texture). The output is a feature map.
2. **Activation (ReLU):** keeps strong responses and sets weak ones to zero.
3. **Pooling or striding:** shrinks the map, keeping the strongest responses.
4. **Stack many layers:** edges become corners and curves, then wheels and windows, then a whole car.

The same filter is used everywhere in the image, so it finds a wheel on the left or the right. The filters are learned during training, not designed by anyone. YOLO and EasyOCR use CNNs, and CLIP's image side is a related vision network. Your LSTM is a different kind of network, built for sequences.

Say: "A CNN slides small learned filters across the picture to detect patterns, stacking layers so edges build up into parts and then whole objects."

What is OpenCV

OpenCV is a toolbox of functions for working with images and video. It does the handling around YOLO, and it does not detect cars itself.

Job	What OpenCV does	Where
Decode images	Turns downloaded JPEG bytes into a grid of pixel numbers	`load_frame()` in `src/cv_module.py`
Read video	Pulls frames and timestamps from a live stream	Live monitor, speed check
Draw on frames	Draws the boxes, labels and speeds	Annotated frames
Crop and resize	Cuts out a car and enlarges it 4 times for plate reading	Plates, evidence pictures
Convert color	BGR for OpenCV, RGB for display, HSV for color naming	Vehicle color, display
Measure pixels	Checks pixel values, such as the "mostly white" test	Camera-unavailable check

OpenCV treats an image as a NumPy array: height by width by 3 color values per pixel. The flow is: OpenCV loads the frame, YOLO finds vehicles, OpenCV draws the boxes.

Say: "OpenCV is the library I use for everything around the detector: loading images, reading live video frames, cropping, color conversion, and drawing the boxes. YOLO does the actual detection."

What is OCR

OCR stands for Optical Character Recognition: reading text out of an image, turning pixels that look like letters and numbers into text a computer can use. The program finds regions that look like text, a neural network recognizes each character or word, and it outputs text with a confidence score.

Your project uses it to try to read license plates (ANPR), with the **EasyOCR** library in `src/anpr.py`. It crops the picture to the car, enlarges the crop 4 times, reads any text, keeps strings of 4 to 12 letters or digits, and accepts one only if confidence is at least 0.35. If no plate is read, the challan says "NOT READ".

It usually fails here because the cameras are far away and low-resolution, so a plate is a few pixels wide or hidden. In tests, real footage mostly returned "not readable", which is the expected result, not a bug. On a clean test image with clear text it reads correctly.

Say: "OCR reads text from images. I use it to try to read plates, but at this camera resolution it rarely succeeds, so the app reports 'not read' honestly instead of guessing."

C3. How the live feed is captured

The cameras belong to **Caltrans**, California's highway department. Caltrans captures the video; your app only fetches it. There are two kinds of feed.

	Captured by	Fetched with	Used for
Snapshot (still image, about 1 per minute)	Caltrans camera	`requests` + OpenCV	Congestion readings, grid, route
Video to watch	Caltrans camera	hls.js, in your browser	The live video panel
Video to analyze	Caltrans camera	OpenCV `VideoCapture`	Tracking, speed, challans

- **Camera list:** `src/live_cameras.py` downloads Caltrans's public camera list (a JSON file per district). Each entry has the camera's name, coordinates, snapshot URL and video URL.
- **Snapshots:** `load_frame()` downloads the JPEG and decodes it with OpenCV.
- **Video:** most cameras publish an HLS stream (a `playlist.m3u8` file pointing to short video chunks) on `wzmedia.dot.ca.gov`. The hls.js player in your browser plays it directly from Caltrans. The live monitor and speed check read frames from it on the server with OpenCV (FFmpeg underneath), using the stream's own timestamps.

Snapshots refresh only about once a minute, so the congestion reading is near-live. Many video streams are offline at any moment.

C4. How the density ratio (frame area) is calculated

It is simple arithmetic on the box coordinates YOLO returns, in `analyze_frame` in `src/cv_module.py`.

```
box area      = (x2 - x1) x (y2 - y1)
density_ratio = sum of all vehicle box areas / (frame width x frame height)      (capped
 at 1.0)
```

Worked example, a 1280 by 720 frame (921,600 pixels) with three vehicles:

Vehicle	Box width x height	Area
Car 1	80 x 50	4,000
Car 2	60 x 40	2,400
Truck	120 x 90	10,800
Total		17,200

Density = 17,200 / 921,600 = 0.0187, about 1.9 percent of the picture, which is under 0.05, so the level is **Low**. The thresholds are: under 0.05 Low, 0.05 to 0.15 Moderate, 0.15 to 0.30 High, 0.30 and above Severe.

Things to know: overlapping boxes are counted twice; a close truck covers more pixels than ten distant cars (perspective); the whole frame is the denominator, including sky and the timestamp bar, which is why the thresholds are small numbers; it is a proxy for congestion, not a direct measurement, which is why it was checked against hand-labelled frames.

Say: "I add up the area of every vehicle box (width times height) and divide by the area of the whole image. That fraction is the density, and it maps to Low, Moderate, High or Severe."

C5. How vehicle type, color, make and model are found

What	Tool used	Where
Car, truck, bus or motorcycle	**YOLOv8s**, the same detector	`src/cv_module.py`
Body type (sedan, SUV, pickup truck, hatchback, minivan, bus, truck, motorcycle)	**CLIP**	`src/vehicle_attributes.py`
Color	**K-means clustering** of the car's pixels (shadows and glare filtered out), then the dominant shade is named	`src/vehicle_attributes.py`
Make and model	**CLIP** again, strictly gated	`src/vehicle_attributes.py`
Number plate	**EasyOCR**	`src/anpr.py`

How CLIP picks the body type. CLIP is a model trained on huge numbers of images paired with text. The app gives it the car's crop and a list of descriptions such as "a sedan car", "an SUV", "a pickup truck". CLIP scores how well the picture matches each one and the best match wins. This is zero-shot classification: the model was never trained on traffic cameras or on your labels. The app reports a type only when confidence is at least 25 percent.

Limits. Body type is usually 30 to 55 percent confident on clear daytime crops and weaker on small or night images. Color can be confused by shadows. Make and model rarely works: the median car on these cameras is only 19 pixels wide, so crops under 70 pixels are refused and a model is reported only above 30 percent confidence.

C6. The dataset: what is in it and why it was used

The **Metro Interstate Traffic Volume** dataset (UCI Machine Learning Repository) is in `data/Metro_Interstate_Traffic_Volume.csv`. It has **48,204 hourly rows**, from 2 October 2012 to 30 September 2018, for traffic on Interstate 94 near Minneapolis and St. Paul, Minnesota, with the weather at each hour.

Column	What it is	Example
`date_time`	the hour	2014-05-12 20:00
`traffic_volume`	**vehicles counted that hour (what the models predict)**	2,690
`temp`	temperature in Kelvin (the code converts to Celsius)	286.6
`rain_1h`, `snow_1h`	rain and snow in the last hour, mm	0.25
`clouds_all`	cloud cover, percent	68
`weather_main`	the weather type	Clouds, Clear, Rain, Snow, Mist
`holiday`	the holiday's name, or "None"	Columbus Day

Numbers to know: median volume about 3,380 vehicles an hour and a maximum of 7,280; the daily pattern is strong (about 370 at 3 AM, 4,740 at 7 AM, 5,664 at 4 PM, 1,469 at 11 PM); holidays are rare (61 of 48,204 hours); a few bad rows exist (10 readings with a temperature of 0 K).

Why this dataset: it is the right shape (hourly traffic volume plus weather and time); it is large and spans years, so the models can learn daily, weekly and seasonal patterns and be tested properly on the newest data; it is free and public, so anyone can reproduce your results; and it lets you show real ML technique (time-based splitting, lag features, comparing three models).

The honest limit: it is Minnesota data, not California. For the California cameras the historical model is a time-of-day and weather baseline, not a local prediction. The fix would be California sensor data (PeMS).

C7. How the three ML models work, are trained and are used

All three models take the same **17 input features** and output one number: the traffic volume for that hour. Features: hour, day of week and month (each as sin and cos), weekend flag, temperature, rain, snow, clouds, holiday flag, weather code, and traffic 1 hour ago, 3 hours ago, 24 hours ago, and the 3-hour average. Features are standardized before training.

Model	How it makes a prediction	Notes
Linear Regression	A weighted sum of the features: `volume = w1*f1 + ... + w17*f17 + bias`. Training finds the weights that make squared errors smallest.	Instant to train; explainable; only straight-line relationships.
SVR	Fits a flexible curved surface with an RBF kernel: it blends the values of similar important training hours, weighted by similarity.	`C=100, epsilon=10, gamma=auto`, tuned with RandomizedSearchCV and TimeSeriesSplit. Trained on 8,000 sampled rows because training slows quickly with more rows.
LSTM	A neural network that reads the previous 24 hours of features and learns patterns over time through "gates" that decide what to remember and forget.	2 stacked LSTM layers of 64 units, 20 percent dropout, then a small 64 to 32 to 1 output network.

How the LSTM is trained (`src/train_dl.py`)

1. **Split by time:** oldest 70 percent train, next 15 percent validation, newest 15 percent test.
2. **Scale** features and the target using only the training part.
3. **Build sequences:** each example is hours 1 to 24, and the answer is the volume at hour 25; then hours 2 to 25 predict hour 26, and so on.
4. **Training loop** (up to 40 rounds called epochs, batches of 128): predict, measure the error with **MSE** loss, use backpropagation to find how much each weight caused the error, and let the **Adam** optimizer nudge every weight a small step (learning rate 0.001).
5. **Validation each round,** with **early stopping** after 5 rounds without improvement; the best weights are kept and reloaded.
6. **Final test** on the newest 15 percent, converted back to real vehicle counts, then MAE, RMSE and R2 are calculated, and the model is saved as `models/lstm.pt`.

Scores vary slightly between runs because the weights start from random numbers, which is why the README says "about 0.977".

Say: "I trained the LSTM on sliding 24-hour windows of the features to predict the next hour's traffic, minimizing squared error with the Adam optimizer, and used a validation set with early stopping to keep the best version. I tested it only on the newest 15 percent of the data."

Where each model is used in the app

The app asks all three for a prediction every time, then uses them in different places.

Where in the app	Which model	How it is used
"All three models" table (Single camera)	All three	Shows each model's R2, predicted volume and level side by side.

Where in the app	Which model	How it is used
"Model driving the forecast" radio	Your choice (LSTM by default)	Sets the historical congestion label shown beside the camera reading; draws the 24-hour forecast; powers "When should I leave?".
Forecast error band	The chosen model's own error	Shaded band: LR plus or minus 334, SVR plus or minus 254, LSTM plus or minus 203 vehicles an hour.
Anomaly warning	The chosen model	Compares its label with the live camera's reading.
Monitoring grid and Route	LSTM (fixed)	One historical label per run, used for the overall reading.

The differences between them are small (R2 0.948 against 0.977), because the lag features already give the simpler models most of the information.

Say, about 60 seconds: "Machine learning does the prediction side. I trained three models, Linear Regression, SVR and an LSTM, on 48,000 hours of traffic and weather data to predict how many vehicles to expect at a given time. I used sin and cos for the hour so midnight sits next to 11 PM, and I added traffic from 1, 3 and 24 hours earlier, which gave the biggest gain. I split the data by time, so the models are tested only on the future they never saw. The LSTM scored highest, with an R-squared of about 0.977, and the simpler models were close behind. The app turns the prediction into a congestion level, a 24-hour forecast, and an unusual-congestion warning, and fuses it with what the live camera shows. The weakness is that this dataset is from Minnesota, so for California it is a time-of-day and weather baseline, and the live camera does the real work."

C7b. The ML part of the project: what the three models represent, and the maths behind them

What "the ML part" means here

The project has two halves. The **machine learning half** answers: "given the time, the weather and recent traffic, how many vehicles should be on this road?" The **computer vision half** answers: "how many vehicles are on the road right now?" The two are then fused.

The ML half is **supervised regression**: the models are shown many examples where the inputs (time, weather, recent traffic) and the correct answer (the real traffic volume) are both known, and they learn the relationship. Then they predict volumes for hours they have never seen.

```
Raw CSV (48,204 hours)
   -> data_loader.py: build the 17 features (sin/cos time, weather, lags)
   -> split by time: 70% train / 15% validation / 15% test
   -> train LR, SVR, LSTM on the training part
   -> score each on the unseen test part (MAE, RMSE, R2)
   -> save the models in models/
   -> live: build the 17 features for "now", ask all three models, volume -> label
```

The three models are a ladder, each adding one idea

	Linear Regression	SVR	LSTM
What it assumes	volume is a weighted sum of the features	volume is a smooth curve of the features	volume depends on the pattern of the last 24 hours
What it can capture	straight-line effects only	curved effects and interactions	curved effects plus how things change over time

	Linear Regression	SVR	LSTM
What it needs	almost nothing	choosing C, epsilon, gamma	many choices: layers, size, learning rate, stopping
Training cost	instant	slow with many rows (so 8,000 sampled)	the heaviest
Explainable?	yes, one weight per feature	partly	hard
Score (R2)	0.948	0.960	about 0.977

Using all three shows what each added idea buys. The gaps are small because the lag features (traffic 1, 3 and 24 hours ago) already give even the simple model most of the information.

How each prediction is calculated

Linear Regression. A straight weighted sum:

```
volume = w1*hour_sin + w2*hour_cos + ... + w17*rolling_mean_3h + bias
```

Training picks the 17 weights and the bias that make the sum of squared errors on the training data as small as possible (ordinary least squares, which has a direct solution, `w = (X^T X)^-1 X^T y`). A positive weight on `lag_1h` means "more traffic an hour ago, more traffic now". Illustrative only, with made-up numbers: if the standardized `lag_1h` is +1.2 and its weight is 900, that feature alone adds about 1,080 vehicles to the prediction.

SVR (Support Vector Regression). Instead of one line, it predicts by comparing the new hour with the most informative training hours (the "support vectors"):

```
volume = sum over support vectors of ( a_i * K(x_i, x) ) + bias
K(x_i, x) = exp( -gamma * distance(x_i, x)^2 )
```

`K` is the RBF kernel: it equals 1 when two hours are identical and falls toward 0 as they get further apart, so the prediction is a similarity-weighted blend of the training hours that look like this one. Two settings shape the fit: `epsilon = 10` means errors under 10 vehicles cost nothing (so it does not chase noise), and `C = 100` sets how strongly larger errors are penalized. Only the support vectors (hours near or outside the epsilon margin) get a non-zero weight `a_i`.

LSTM (Long Short-Term Memory). A neural network that reads the 24 hours one step at a time, keeping a memory. At each hour it uses three small "gates", each a learned weighted sum squashed to a value between 0 and 1:

```
forget gate   f = sigmoid( W_f . [h_prev, x] + b_f )
input gate    i = sigmoid( W_i . [h_prev, x] + b_i )
candidate     g = tanh(    W_g . [h_prev, x] + b_g )
memory        c = f * c_prev + i * g
output gate   o = sigmoid( W_o . [h_prev, x] + b_o )
output        h = o * tanh(c)
```

Here `x` is this hour's 17 features, `h_prev` is the previous hour's output, and `c_prev` is the memory so far. The **forget gate** decides what to drop from memory, the **input gate** decides how much of the **candidate** (the new information) to store, the **memory** line updates the memory, and the **output gate** decides what to pass on as `h` to the next hour.

After the 24th hour, the last output `h` goes through a small network (64 to 32 to 1) to give the predicted volume. All the `W` and `b` numbers are what training adjusts. Two LSTM layers are stacked, with 64 units each.

How training finds the numbers

All three learn by reducing the same kind of error, squared prediction error, on the training data: Linear Regression solves for the weights directly; SVR solves an optimization that finds the support vectors; the LSTM improves its weights gradually with backpropagation and the Adam optimizer, checking a validation set after every round and stopping early when it stops improving. In every case, the final score comes from the test set the model never saw.

From volume to a congestion level

A predicted volume is turned into Low, Moderate, High or Severe by comparing it with the **quartiles of the historical volumes** (the 25th, 50th and 75th percentiles). Below the 25th percentile is Low, between the 25th and 50th is Moderate, between the 50th and 75th is High, and above the 75th is Severe. That label is what gets fused with the live camera's label.

What counts as "ML" in this project, and what does not

Part	Type	Trained by you?
Linear Regression, SVR, LSTM	Supervised regression (the ML half)	Yes
YOLOv8 detector	Deep learning (CNN), pretrained	No; tuned and evaluated by you
CLIP (vehicle type)	Deep learning (vision-language), pretrained	No
EasyOCR (plates)	Deep learning, pretrained	No
K-means (vehicle color)	Unsupervised clustering	Fits itself to each picture
Fusion rule, tracker, speed model, thresholds	Hand-designed logic and geometry	Written by you

Say: "The machine learning part of my project is the three prediction models. They are supervised regression models that learn from 48,000 hours of traffic and weather to predict how many vehicles to expect. Linear Regression is a weighted sum, SVR is a similarity-based curve, and the LSTM reads the previous 24 hours with memory gates. I trained and tested all three the same way, split by time, and compared them with MAE, RMSE and R-squared. The computer vision half then checks what the camera actually shows, and the two are fused."

If asked "why not just use the LSTM?": "Because comparing three shows what each idea adds. The simple linear model is almost as good once the right features are there, and it is explainable. The LSTM gives the best score, so it is the default."

C8. The decimal numbers: what they mean and how they are calculated

Where you see it	Example	What it means
Labels on the boxes in the photo	`car 0.66`	**Confidence**: how sure the detector is (0.66 = 66 percent). The app keeps detections down to 0.08.
Image density ratio	`0.0179`	The share of the picture covered by vehicle boxes (about 1.8 percent).
Model names	`LSTM (R2=0.977)`	**R2**: how well the model fits held-out data (1.0 is perfect). It belongs to the historical model, not the camera part.
Forecast chart band	plus or minus 203 vehicles an hour	The model's typical error (MAE) on held-out data.
Input boxes	`45.00`, `65.00`	Numbers in input boxes, shown with two decimals.

Where you see it	Example	What it means
Live monitor status	`calibration scale x2.26`	How much speeds were multiplied to match typical traffic speed. A big number means speeds are mostly relative.
Camera model	`fit quality 0.96`	How well the camera geometry fit the traffic (the app refuses below 0.7).
Counting error	`7.43 to 4.55`	Average miss in vehicles per frame (MAE), before and after tuning.
Label scores	`0.32 to 0.62`	**Balanced accuracy**: average success per class. It matters because 74 percent of frames are Low.

Formulas

```
Single camera:    badge = live camera label; historical label shown as context (route
overview takes the more severe of the two)
MAE               = average of |predicted - actual|
RMSE              = square root of the average of (predicted - actual)^2
R2                = 1 - (sum of squared errors) / (total variation around the average)
hour_sin, hour_cos = sin(2*pi*hour/24), cos(2*pi*hour/24)
Recall (one class) = frames of that class labelled correctly / all frames of that class
Balanced accuracy  = average of the recall of each class
F1 (one class)     = 2 x precision x recall / (precision + recall)
Distance ahead     = focal_length x camera_height / (y - horizon)
Sideways position  = (x - image_centre) x distance / focal_length
Speed (mph)        = slope of ground position over time (m/s) x 2.237
Scale factor       = typical traffic speed / median tracked speed
Macro-F1          = the average of F1 over the four classes
```

How the ML scores are computed: the models are trained on the oldest 70 percent, tuned on the next 15 percent, and scored on the newest 15 percent, which they have never seen, using MAE, RMSE and R2. A score of 0.977 does not mean "right 97.7 percent of the time"; it measures how closely predictions follow real traffic on unseen hours.

C9. Why California, and why not India

All live cameras are **Caltrans** (California). Caltrans publishes about 3,400 camera images, live video for most of them, and coordinates, all free with no key or registration. That is what let the app do live detection, routes and video.

I checked India. I found no Indian government API that serves live traffic camera feeds. The government open-data portal (data.gov.in) has traffic-junction CCTV records, but they are one-time summary counts (cameras per city), not a live feed. Websites listing Indian CCTV streams are unofficial, and some warn the streams may be outdated or unauthorized. Live camera access in India usually sits behind city traffic-management control rooms. For training data, I found only small datasets that do not match hourly highway volume plus weather (a Pune intersection count dataset, a Bangalore travel-time dataset, and an IEEE DataPort junction-count set).

```
data.gov.in:  https://tn.data.gov.in/resource/cctv-cameras-traffic-junctions-2019-2020
Pune counts:  https://data.mendeley.com/datasets/xnf2k6n288
Bangalore:    https://groups.google.com/g/datameet/c/NDT9b0nmVgM/m/l7-Tvkv9CQAJ
IEEE DataPort: https://ieee-dataport.org/comment/7159
```

What works outside California: the Upload image and Snapshot URL tabs work for any traffic photo anywhere; route and place search work worldwide but have no cameras to check; the live video, grid and Live e-Challan tabs are California only. Uploaded images use California time for the historical part.

Say: "I used California because Caltrans publishes free, open, machine-readable live cameras. I looked at India and could not find a public live-camera feed; those are usually held by city traffic control rooms. The detection, fusion and routing do not depend on the country, so adding India would mean getting access to a camera source from a city authority. It is listed as future work, and the Upload tab already works on any Indian traffic photo."

Describe it as "I could not find one", not "none exists".

C10. What the project is built around

Congestion is the main feature. **Speed** is an experimental extra.

1. **Congestion prediction** is the heart: machine learning predicts expected traffic from time and weather, computer vision measures what the live camera shows, and the two are shown together (the badge follows the camera) (Low, Moderate, High, Severe).
2. **Built on that result:** the forecast and "when should I leave", the monitoring grid, and the route map colored by congestion.
3. **Speed and challans** (the Live e-Challan tab) are an additional, experimental part. They show what the same vision pipeline can do, but they are simulated and the speeds are not validated against radar.

Do not call speed a main feature: it is the least reliable part, and the numbers you can defend (counting error, R2 of 0.977) belong to the congestion part.

Say: "The core of the project is congestion prediction: machine learning plus computer vision, shown together (the badge follows the camera). Everything else builds on it. I also added a speed and challan pipeline as an experimental extension, and I am open that its speeds are estimates."

C11. What makes this project different

Say, about 30 seconds: "Most existing traffic systems do one of two things. Navigation apps like Google Maps use phone and sensor data that only the company has. Many student projects either predict traffic from a historical dataset, or count cars in a picture. Mine combines both on real, free, public camera feeds. It predicts what traffic should be from time and weather, checks what the camera actually shows, and reports the camera's reading with the prediction beside it, flagging when the camera shows far more traffic than expected. I also measured the vision part on frames I labelled, and I am open about where it falls short."

Three points to back it up:

1. **It combines two signals, and I can say why.** A weighted average failed on a gridlocked image; taking the worse of the two made empty California roads read High; so the badge now follows the camera and history is context.
2. **It works on real, messy live data.** It found real failures (about 1 in 5 cameras serves an "unavailable" image, which the first version read as an empty road) and fixed them.
3. **It measures itself and goes further.** An evaluation on 162 labelled frames, ideas I tried and rejected, and then routes, live tracking, speed estimates and a simulated challan pipeline, with the limits stated.

If asked "Google Maps already does this": "Yes, Google has far better data. This project is not trying to compete with that. It shows how to build a congestion system from only free public data, how to combine ML and computer vision, and how to measure how well it works."

Do not say: that it is more accurate than existing systems, that the speeds are accurate, that it is completely new, or that it can identify car makes. Close with the limit before they ask: the historical model was trained on Minneapolis data, so for California it is a time-of-day baseline.

C12. Which cameras to use during the presentation

Use cameras in **District 7 (Los Angeles)**, where everything was tested, and run `python demo_check.py` about 20 minutes before you present. It lists the cameras with the most traffic and a working video stream at that moment. Cameras go offline, so treat these as defaults, not guarantees.

Part of the demo	Camera or route to use	Why
Single camera	Any I-5 or I-405 camera in District 7	Busy freeway shots show more detections. The I-110 Avenue 26 default works but is often quiet.
Monitoring grid	District 7, filter `I-405` or `US-101`, 6 cameras	Freeway cameras usually have traffic. Leave an "unavailable" card in as a talking point.
Route	Search places: **Burbank Airport to Santa Monica Pier**	Tested: 40.7 km, 16 cameras, visually strong.
Live e-Challan	**I-5 : (37) Olive, Burbank** (the default)	The camera that calibrated speeds well in tests.

Backups for the Live e-Challan tab: the I-5 cameras (17) South of I-10, (31) Zoo Drive and (40) North of Burbank Blvd, all District 7. Their streams worked in earlier tests but were not tested in the live monitor at a busy hour.

Avoid District 4 (a large share of its cameras served the unavailable image in my sample), cameras with a sideways view (the monitor will say speeds are not reliable), and quiet or night cameras. Best windows are California 7 to 9 AM or 4 to 6 PM, which is 7:30 to 9:30 PM or 4:30 to 6:30 AM India time.

C13. Script to read: the computer vision presentation (about 5 minutes)

Lines marked DEMO are cues for what to click.

Opening. Good morning. My project is a traffic congestion system, and I will focus on the computer vision part. The question it answers is simple: can a program look at a live highway camera and tell how congested the road is?

The problem. Predicting traffic from history alone misses what is happening right now, like an accident. A camera shows the road as it is. So my system uses computer vision to read live cameras, and combines that with a machine learning model that knows what traffic is normally like.

Detecting vehicles. DEMO: Single camera, Fetch live snapshot. Here is a real Caltrans camera in Los Angeles. The boxes are from YOLOv8, an object detector. It is a convolutional neural network: early layers find edges, later layers find parts like wheels and windows, and the final layers predict a box and a class for each vehicle, all in one pass. That is why it is fast enough for live video. YOLO is pretrained. I did not train it. What I did was tune it and test it.

From boxes to congestion. I add up the area of all the vehicle boxes and divide by the area of the image. That is the density ratio. Under 0.05 is Low, up to 0.15 Moderate, up to 0.30 High, and above that Severe.

Measuring it honestly. A model without a measured accuracy is just a claim, so I labelled 162 real frames from all 12 districts, with vehicle counts and congestion levels. That showed two real problems. First, about one in five cameras was serving a "Temporarily Unavailable" image. The detector saw no cars and reported an empty road. Now the app recognizes that image and says the camera is unavailable. DEMO: Monitoring grid, Load grid, point at an unavailable card. Second, at its default setting, the detector missed about 7 vehicles per frame. Lowering the confidence threshold cut the counting error from 7.4 to 4.6. I also tried a bigger model, higher resolution, a learned classifier and sliced detection. None helped enough, so I kept the simpler version.

Tracking and speed. DEMO: Live e-Challan, Start monitoring. Now the live video. The app tracks each car from frame to frame. To estimate speed I had a problem: these cameras have no calibration data. So I fit a camera model from the traffic itself. For a camera looking along a flat road, a car's position in the image tells you how far away it is. With that, I convert movement on screen into movement on the road. I tested the math on synthetic cars of known speed, and it is accurate to within 4 percent. But I have not validated it against radar, so I treat the speeds as estimates, and the app refuses to issue anything if it cannot calibrate a camera. Any challan you see is simulated and labelled that way.

Other vision tools. I also use CLIP, a vision-language model, to guess the vehicle's body type, and color clustering for its color. OCR tries to read plates, but at this resolution the median car is only 19 pixels wide, so it usually cannot, and the app says so honestly.

Limits. Three honest limits. My evaluation set is small, daytime only, and labelled by hand. The speeds are unvalidated. And I did not train the detector. Fine-tuning it on traffic footage is the next step, along with California sensor data for the prediction side.

Close. To summarize: I used a pretrained detector, built the measuring and tracking around it, found real failures, fixed them, and kept only what the evidence supported. Thank you. I am happy to take questions.